Kernel: SageMath 10.7

In [2]:

```
from itertools import combinations

# ============================================================
# Bases
# ============================================================

# Basis of Λ³(R⁶) (20 elements)
basis3 = list(combinations(range(6), 3))

# Basis of Λ²(R⁶) (15 elements)
basis2 = list(combinations(range(6), 2))

# Fast lookup: pair -> row index
basis2_pos = {pair: m for m, pair in enumerate(basis2)}

EPS = 1e-12


# ============================================================
# Build α from 20 coefficients
# ============================================================

def alpha_from_coeffs(coeffs):
    """
    Input:
        coeffs = list of 20 numbers

    Output:
        dictionary representing α ∈ Λ³(R⁶)
    """
    assert len(coeffs) == 20, f"Need 20 coefficients, got {len(coeffs)}"

    return {
        basis3[m]: RR(coeffs[m])
        for m in range(20)
        if abs(coeffs[m]) > EPS
    }


# ============================================================
# Build α from decomposable terms
# ============================================================

def alpha_from_wedge(*triples):
    """
    Each argument is (u,v,w)

    u,v,w are vectors in R^6.

    Returns:
        α = Σ (u∧v∧w)
    """

    alpha = {idx: RR(0) for idx in basis3}
```

```
    for (u, v, w) in triples:

        for (i, j, k) in basis3:

            det = (
                u[i]*(v[j]*w[k] - v[k]*w[j])
                - u[j]*(v[i]*w[k] - v[k]*w[i])
                + u[k]*(v[i]*w[j] - v[j]*w[i])
            )

            alpha[(i, j, k)] += det

    return {
        k: v
        for k, v in alpha.items()
        if abs(v) > EPS
    }


# ============================================================
# Interactive input
# ============================================================

def alpha_from_input():

    print("\nBasis order:\n")

    for m, idx in enumerate(basis3):
        name = "e" + "".join(str(i + 1) for i in idx)
        print(f"{m+1:2d}: {name}")

    print("\nEnter 20 coefficients separated by spaces:\n")

    coeffs = list(map(float, input().split()))

    return alpha_from_coeffs(coeffs)


# ============================================================
# Contraction i_{e_s}(α)
# ============================================================

def contraction(alpha, s):

    result = [RR(0)] * 15

    for (i, j, k), coeff in alpha.items():

        if abs(coeff) < EPS:
            continue

        if s == i:
            result[basis2_pos[(j, k)]] += coeff

        elif s == j:
            result[basis2_pos[(i, k)]] -= coeff
```

```
        elif s == k:
            result[basis2_pos[(i, j)]] += coeff

    return result


# ============================================================
# Build ϕ_α : R⁶ → Λ²(R⁶)
#
# Returns the 15×6 matrix
# ============================================================

def phi_matrix(alpha):

    cols = [contraction(alpha, s) for s in range(6)]

    return matrix(RR, list(map(list, zip(*cols))))


# ============================================================
# Rank of ϕ_α
# ============================================================

def phi_rank(alpha):

    M = phi_matrix(alpha)

    return M.rank()


# ============================================================
# Analysis
# ============================================================

def analyze(alpha):

    print("\nNonzero coordinates of α:\n")

    for idx, coeff in sorted(alpha.items()):

        name = "e" + "".join(str(i + 1) for i in idx)

        print(f"{name} : {coeff}")

    M = phi_matrix(alpha)

    print("\nϕ_α =\n")
    print(M)

    r = M.rank()

    print("\nrank(ϕ_α) =", r)

    return r
```

```
# # Way 1: from user input (prompts for 20 numbers)
# alpha = alpha_from_input()
# analyze(alpha)

# # Way 2: from 20 coefficients directly
# alpha = alpha_from_coeffs([1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1])
# analyze(alpha)

# # Way 3: from wedge products of vectors (most natural later)
# u = [1,0,0,0,0,0]
# v = [0,1,0,0,0,0]
# w = [0,0,1,0,0,0]
# alpha = alpha_from_wedge((u,v,w))
# analyze(alpha)

# # Way 3 with two summands
# alpha = alpha_from_wedge(
#     ([1,0,0,0,0,0], [0,1,0,0,0,0], [0,0,1,0,0,0]),  # e1∧e2∧e3
#     ([0,0,0,1,0,0], [0,0,0,0,1,0], [0,0,0,0,0,1])   # e4∧e5∧e6
# )
# analyze(alpha)
```

```
# ============================================================
# Generic basis builder
# ============================================================

def basis_k(k):
    return list(combinations(range(6), k))


# ============================================================
# Sign of permutation
# ============================================================

def permutation_sign(seq):
    """
    Returns sign of permutation needed
    to sort seq into increasing order.
    """

    seq = list(seq)

    sign = 1

    for i in range(len(seq)):
        for j in range(i + 1, len(seq)):
            if seq[i] > seq[j]:
                sign *= -1

    return sign
```

```
# ============================================================
# Wedge product
# ============================================================

def wedge(A, B):
    """
    A,B represented as dictionaries

    Example:

        {(0,1):2}
        means 2 e12

    Returns A∧B.
    """

    result = {}

    for I, a in A.items():

        for J, b in B.items():

            K = I + J

            # repeated index => zero

            if len(set(K)) < len(K):
                continue

            sign = permutation_sign(K)

            Ksorted = tuple(sorted(K))

            result[Ksorted] = (
                result.get(Ksorted, RR(0))
                + sign*a*b
            )

    return {
        k: v
        for k, v in result.items()
        if abs(v) > EPS
    }
```

```
# ============================================================
# Basis of Λ⁵(R⁶)
# ============================================================

basis5 = list(combinations(range(6),5))

basis5_pos = {
    idx:m
    for m,idx in enumerate(basis5)
}
```

```
def contraction_form(alpha,s):
    """
    returns i_{e_s}(alpha)
    as a dictionary
    """

    result = {}

    for (i,j,k), coeff in alpha.items():

        if s == i:
            result[(j,k)] = result.get((j,k),RR(0)) + coeff

        elif s == j:
            result[(i,k)] = result.get((i,k),RR(0)) - coeff

        elif s == k:
            result[(i,j)] = result.get((i,j),RR(0)) + coeff

    return result


def beta_s(alpha,s):
    """
    β_s=(i_{e_s}α)∧α

    returns a 5-form
    """

    return wedge(
        contraction_form(alpha,s),
        alpha
    )
```

```
alpha = {
    (0,1,2):1,
    (3,4,5):1
}
for s in range(6):
    print(s+1, beta_s(alpha,s))
```

Out[7]:
```
1 {(1, 2, 3, 4, 5): 1.00000000000000}
2 {(0, 2, 3, 4, 5): -1.00000000000000}
3 {(0, 1, 3, 4, 5): 1.00000000000000}
4 {(0, 1, 2, 4, 5): 1.00000000000000}
5 {(0, 1, 2, 3, 5): -1.00000000000000}
6 {(0, 1, 2, 3, 4): 1.00000000000000}
```

```
# ============================================================
# Missing index of a basis 5-form
#
# Example:
#   (1,2,3,4,5) -> 0
#   (0,2,3,4,5) -> 1
#
```

```
# Since every basis element of Λ⁵(R⁶) misses exactly one index.
# ============================================================

def missing_index(five_tuple):

    for i in range(6):
        if i not in five_tuple:
            return i


# ============================================================
# Convert a 5-form into a vector in R⁶
#
# This implements Hitchin's map
#
#     A : Λ⁵(R⁶) → (R⁶)*
#
# using the volume form
#
#     vol = e123456.
#
# Example:
#
#     e23456 ↦ e¹
#     e13456 ↦ -e²
#
# The sign is determined by:
#
#     e^m ∧ beta = ± vol
# ============================================================

def fiveform_to_vector(beta):

    v = vector(RR, [0]*6)

    for idx, coeff in beta.items():

        # missing basis index
        m = missing_index(idx)

        # sign of
        # (m, i1, i2, i3, i4, i5)
        sign = permutation_sign((m,) + idx)

        v[m] += sign * coeff

    return v


# ============================================================
# Hitchin operator
#
# K_α : R⁶ → R⁶
#
# defined by
#
#     K_α(e_s)
```

```
#       = A( (i_{e_s} α) ∧ α )
#
# Columns are K_α(e₁),...,K_α(e₆).
# ============================================================

def hitchin_operator(alpha):

    cols = []

    for s in range(6):

        # β_s = (i_{e_s} α) ∧ α
        beta = beta_s(alpha, s)

        # convert 5-form to vector
        col = fiveform_to_vector(beta)

        cols.append(col)

    # columns -> matrix
    K = matrix(RR, list(map(list, zip(*cols))))

    return K


# ============================================================
# Hitchin quartic invariant
#
# λ(α) = (1/6) tr(K_α²)
#
# For stable forms:
#
#     K_α² = λ(α) I
#
# ============================================================

def hitchin_lambda(alpha):

    K = hitchin_operator(alpha)

    return (K*K).trace()/6


# ============================================================
# Stable orbit classification
#
# λ > 0  : positive stable orbit
# λ < 0  : negative stable orbit
# λ = 0  : degenerate orbit
# ============================================================

def stable_type(alpha):

    lam = hitchin_lambda(alpha)

    if lam > EPS:
        return "positive stable"
```

```
    elif lam < -EPS:
        return "negative stable"

    else:
        return "degenerate"


# ============================================================
# Convenience function
# ============================================================

def analyze_hitchin(alpha):

    K = hitchin_operator(alpha)

    print("\nK_alpha =\n")
    print(K)

    print("\nK_alpha^2 =\n")
    print(K*K)

    lam = hitchin_lambda(alpha)

    print("\nlambda(alpha) =", lam)

    print("\nOrbit type =", stable_type(alpha))

    return K, lam
```

```
# alpha = {
#     (0,1,2):1,
#     (3,4,5):1
# }

# analyze_hitchin(alpha)

# rho = {
#     (0,2,4): 1,
#     (0,3,5):-1,
#     (1,2,5):-1,
#     (1,3,4):-1
# }

# analyze_hitchin(rho)

# rho_positive_stable = {
#     (0, 1, 2): 1.0,
#     (3, 4, 5): 1.0
# }
# print("--- TESTING POSITIVE STABLE ORBIT ---")
# analyze_hitchin(rho_positive_stable)

# rho_negative_stable = {
#     (0, 1, 4): 1.0,
#     (0, 2, 3): -1.0,
```

```
#     (1, 2, 5): 1.0,
#     (3, 4, 5): 1.0
# }
# print("\n--- TESTING NEGATIVE STABLE ORBIT ---")
# analyze_hitchin(rho_negative_stable
# )
```

```
# ============================================================
# Orbit classification in Λ³(R⁶)
# ============================================================

def classify_orbit(alpha):

    r = phi_rank(alpha)

    # O0
    if r == 0:
        return {
            "orbit": "O0",
            "normal_form": "0",
            "real_rank": 0
        }

    # O1
    elif r == 3:
        return {
            "orbit": "O1",
            "normal_form": "e123",
            "real_rank": 1
        }

    # O2
    elif r == 5:
        return {
            "orbit": "O2",
            "normal_form": "e123 + e145",
            "real_rank": 2
        }

    # stable case
    elif r == 6:

        lam = hitchin_lambda(alpha)

        if lam > EPS:

            return {
                "orbit": "O3",
                "normal_form": "e123 + e456",
                "lambda": lam,
                "real_rank": 2
            }

        elif lam < -EPS:
```

```
            return {
                "orbit": "O4",
                "normal_form":
                "e135 - e146 - e236 - e245",
                "lambda": lam,

            }

        else:

            return {
                "orbit": "Singular",
                "normal_form": "lambda = 0 locus",
                "lambda": 0
            }

    else:

        return {
            "orbit": "Unknown",
            "rank_phi": r
        }


def analyze_orbit(alpha):

    info = classify_orbit(alpha)

    print("\nOrbit classification\n")

    for k,v in info.items():
        print(f"{k} : {v}")

    return info
```

```
# ============================================================
# Orbit classification in Λ³(R⁶)
# ============================================================

def classify_orbit(alpha):

    r = phi_rank(alpha)

    # O0
    if r == 0:
        return {
            "orbit": "O0",
            "normal_form": "0",
            "real_rank": 0
        }

    # O1
    elif r == 3:
        return {
            "orbit": "O1",
            "normal_form": "e123",
```

```
            "real_rank": 1
        }

    # O2
    elif r == 5:
        return {
            "orbit": "O2",
            "normal_form": "e123 + e145",
            "real_rank": 2
        }

    # stable case
    elif r == 6:

        lam = hitchin_lambda(alpha)

        if lam > EPS:

            return {
                "orbit": "O3",
                "normal_form": "e123 + e456",
                "lambda": lam,
                "real_rank": 2
            }

        elif lam < -EPS:

            return {
                "orbit": "O4",
                "normal_form":
                "e135 - e146 - e236 - e245",
                "lambda": lam,

            }

        else:

            return {
                "orbit": "Singular",
                "normal_form": "lambda = 0 locus",
                "lambda": 0
            }

    else:

        return {
            "orbit": "Unknown",
            "rank_phi": r
        }


def analyze_orbit(alpha):

    info = classify_orbit(alpha)

    print("\nOrbit classification\n")
```

```
    for k,v in info.items():
        print(f"{k} : {v}")

    return info
```

```
alpha = {
    (0,1,2):1
}

analyze_orbit(alpha)


alpha = {
    (0,1,2):1,
    (3,4,5):1
}

rho = {
    (0,2,4): 1,
    (0,3,5):-1,
    (1,2,5):-1,
    (1,3,4):-1
}

analyze_orbit(rho)
```

Out[12]:

```
Orbit classification

orbit : O1
normal_form : e123
real_rank : 1

Orbit classification

orbit : O4
normal_form : e135 - e146 - e236 - e245
lambda : -4.00000000000000

{'orbit': 'O4',
 'normal_form': 'e135 - e146 - e236 - e245',
 'lambda': -4.00000000000000}
```

```
# ============================================================
# Standard representatives of all 5 GL₆(R) orbits
# ============================================================

tests = [

    # ------------------------------------------------------
    # O0 : Zero form
    # ------------------------------------------------------
    (
        "O0",
```

```
        {}
    ),

    # ----------------------------------------------------------
    # O1 : e123
    # ----------------------------------------------------------
    (
        "O1",
        {
            (0,1,2): 1
        }
    ),

    # ----------------------------------------------------------
    # O2 : e123 + e145
    # ----------------------------------------------------------
    (
        "O2",
        {
            (0,1,2): 1,
            (0,3,4): 1
        }
    ),

    # ----------------------------------------------------------
    # O3 : Positive stable orbit
    #      e123 + e456
    # ----------------------------------------------------------
    (
        "O3",
        {
            (0,1,2): 1,
            (3,4,5): 1
        }
    ),

    # ----------------------------------------------------------
    # O4 : Negative stable orbit
    #      e135 - e146 - e236 - e245
    # ----------------------------------------------------------
    (
        "O4",
        {
            (0,2,4):  1,
            (0,3,5): -1,
            (1,2,5): -1,
            (1,3,4): -1
        }
    )
]


# ============================================================
# Run all tests
# ============================================================

for expected_orbit, alpha in tests:
```

```
    print("Expected Orbit:", expected_orbit)


    info = analyze_orbit(alpha)

    print("\nReturned Orbit:", info["orbit"])

    if info["orbit"] == expected_orbit:
        print("PASS")
    else:
        print("FAIL")

    print("\n")    #ALL PASSED
```

Out[13]:
```
Expected Orbit: O0

Orbit classification

orbit : O0
normal_form : 0
real_rank : 0

Returned Orbit: O0
PASS


Expected Orbit: O1

Orbit classification

orbit : O1
normal_form : e123
real_rank : 1

Returned Orbit: O1
PASS


Expected Orbit: O2

Orbit classification

orbit : O2
normal_form : e123 + e145
real_rank : 2

Returned Orbit: O2
PASS


Expected Orbit: O3

Orbit classification

orbit : O3
```

```
normal_form : e123 + e456
lambda : 1.00000000000000
real_rank : 2

Returned Orbit: O3
PASS


Expected Orbit: O4

Orbit classification

orbit : O4
normal_form : e135 - e146 - e236 - e245
lambda : -4.00000000000000

Returned Orbit: O4
PASS
```

```
# ============================================================
# Λ³(R⁶) dictionary <-> coordinate vector
# ============================================================

def alpha_to_vector(alpha):

    v = vector(RR, [0]*20)

    for idx, coeff in alpha.items():
        v[basis3.index(idx)] = coeff

    return v


def vector_to_alpha(v):

    alpha = {}

    for m, coeff in enumerate(v):
        if abs(coeff) > EPS:
            alpha[basis3[m]] = RR(coeff)

    return alpha

# ============================================================
# Dual action on basis 1-forms
# ============================================================

def dual_basis_form(A, j):

    form = {}

    for i in range(6):
```

```
        c = A[j,i]

        if abs(c) > EPS:
            form[(i,)] = c

    return form

# ============================================================
# Exterior power Λ³(A)
# ============================================================

def exterior_power_3(A):

    M = matrix(RR, 20, 20)

    for col, I in enumerate(basis3):

        i,j,k = I

        ei = dual_basis_form(A, i)
        ej = dual_basis_form(A, j)
        ek = dual_basis_form(A, k)

        image = wedge(wedge(ei,ej),ek)

        for J, coeff in image.items():

            row = basis3.index(J)

            M[row,col] += coeff

    return M


# ============================================================
# Hitchin quantity K*Ω
# ============================================================

def hitchin_star_alpha(alpha):

    K = hitchin_operator(alpha)

    Lambda3K = exterior_power_3(K)

    v = alpha_to_vector(alpha)

    image = Lambda3K * v

    return vector_to_alpha(image)


# ============================================================
# Hitchin decomposition
# ============================================================

def hitchin_decomposition(alpha):
```

```
    lam = hitchin_lambda(alpha)

    if lam <= EPS:
        raise ValueError("Need positive stable form")

    mu3 = lam^(3/2)

    Kstar_alpha = hitchin_star_alpha(alpha)

    v1 = alpha_to_vector(alpha)
    v2 = alpha_to_vector(Kstar_alpha)

    alpha_part = (mu3*v1 + v2)/(2*mu3)
    beta_part  = (mu3*v1 - v2)/(2*mu3)

    return (
        vector_to_alpha(alpha_part),
        vector_to_alpha(beta_part)
    )
```

```
from itertools import combinations

def p(omega, i, j, k):
    key = tuple(sorted([i, j, k]))
    return omega.get(key, 0)

def all_plucker_fixed(omega):
    violations = []

    # Family 1: one shared index — 30 relations
    for i in range(6):
        others = [x for x in range(6) if x != i]
        for j,k,l,m in combinations(others, 4):
            val = (p(omega,i,j,k)*p(omega,i,l,m)
                 - p(omega,i,j,l)*p(omega,i,k,m)
                 + p(omega,i,j,m)*p(omega,i,k,l))
            if val != 0:
                violations.append(('F1', (i,j,k,l,m), val))

    # Family 2: ONE fully antisymmetrized relation — the Pfaffian
    val = (p(omega,0,1,2)*p(omega,3,4,5)
         - p(omega,0,1,3)*p(omega,2,4,5)
         + p(omega,0,1,4)*p(omega,2,3,5)
         - p(omega,0,1,5)*p(omega,2,3,4)
         + p(omega,0,2,3)*p(omega,1,4,5)
         - p(omega,0,2,4)*p(omega,1,3,5)
         + p(omega,0,2,5)*p(omega,1,3,4)
         - p(omega,0,3,4)*p(omega,1,2,5)
         + p(omega,0,3,5)*p(omega,1,2,4)
         - p(omega,0,4,5)*p(omega,1,2,3))
    if val != 0:
        violations.append(('F2', val))

    return violations
```

```
def is_decomposable(omega):
    return len(all_plucker_fixed(omega)) == 0
```

```
test_cases = [
    # --- Category A: Basis Permutations & Index Swaps ---
    # Case 1 (Standard Normal Form)
    {(1, 2, 3): 1.0, (4, 5, 6): 1.0},

    # Case 2
    {(1, 2, 4): 1.0, (3, 5, 6): 1.0},

    # Case 3
    {(1, 3, 5): 1.0, (2, 4, 6): 1.0},

    # Case 4
    {(1, 4, 5): 1.0, (2, 3, 6): 1.0},

    # Case 5
    {(1, 5, 6): 1.0, (2, 3, 4): 1.0},

    # Case 6 (Flipped orientation / Sign verification)
    {(1, 2, 3): -1.0, (4, 5, 6): -1.0},

    # Case 7
    {(1, 3, 6): 1.0, (2, 4, 5): 1.0},

    # --- Category B: Scaled & Weighted Intersections ---
    # Case 8
    {(1, 2, 3): 2.0, (4, 5, 6): 3.0},

    # Case 9
    {(1, 2, 4): 5.0, (3, 5, 6): 2.0},

    # Case 10
    {(1, 3, 5): 1.0, (2, 4, 6): 4.0},

    # Case 11
    {(1, 4, 5): 0.5, (2, 3, 6): 2.0},

    # Case 12
    {(1, 2, 3): -1.0, (4, 5, 6): -1.0},

    # --- Category C: Linear Transformations & Mixed Terms ---
    # Case 13: e123 + e234 + e456
    {(1, 2, 3): 1.0, (2, 3, 4): 1.0, (4, 5, 6): 1.0},

    # Case 14: e123 - e156 + e456
    {(1, 2, 3): 1.0, (1, 5, 6): -1.0, (4, 5, 6): 1.0},

    # Case 15: e125 + e126 + e345 + e346
    {(1, 2, 5): 1.0, (1, 2, 6): 1.0, (3, 4, 5): 1.0, (3, 4, 6): 1.0},

    # Case 16: 2e123 + e145 + 2e456
    {(1, 2, 3): 2.0, (1, 4, 5): 1.0, (4, 5, 6): 2.0},
```

```
    # Case 17: e123 + e124 + e345 + e456
    {(1, 2, 3): 1.0, (1, 2, 4): 1.0, (3, 4, 5): 1.0, (4, 5, 6): 1.0},

    # --- Category D: Highly Dense / Random Setup ---
    # Case 18
    {(1, 2, 3): 1.0, (1, 2, 4): 1.0, (1, 3, 5): 1.0, (4, 5, 6): 1.0},

    # Case 19
    {(1, 2, 3): 3.0, (1, 4, 6): 1.0, (2, 4, 5): 1.0, (4, 5, 6): 3.0},

    # Case 20 (Highly Dense)
    {
        (1, 2, 3): 1.0,
        (1, 4, 5): 1.0,
        (1, 5, 6): 1.0,
        (2, 3, 4): 1.0,
        (2, 4, 6): 1.0,
        (4, 5, 6): 1.0
    }
]
```

```
omega = convert_form(test_cases[18])

alpha_part, beta_part = hitchin_decomposition(omega)

print(alpha_part)
print(beta_part)
```

Out[18]:
```
{(0, 1, 2): 3.00000000000000, (0, 1, 3): 0.333333333333333}
{(0, 1, 3): -0.333333333333333, (0, 3, 5): 1.00000000000000, (1, 3, 4):
1.00000000000000, (3, 4, 5): 3.00000000000000}
```

```
for i, case in enumerate(test_cases, start=1):

    omega = convert_form(case)

    lam = hitchin_lambda(omega)

    if lam <= EPS:
        print(f"\nCASE {i}: skipped")
        continue

    alpha, beta = hitchin_decomposition(omega)

    print("\n====================")
    print("CASE", i)

    print("phi_rank(alpha) =", phi_rank(alpha))
    print("phi_rank(beta)  =", phi_rank(beta))
```

Out[22]:
```
====================
CASE 1
phi_rank(alpha) = 3
```

```
phi_rank(beta)  = 3

====================
CASE 2
phi_rank(alpha) = 3
phi_rank(beta)  = 3

====================
CASE 3
phi_rank(alpha) = 3
phi_rank(beta)  = 3

====================
CASE 4
phi_rank(alpha) = 3
phi_rank(beta)  = 3

====================
CASE 5
phi_rank(alpha) = 3
phi_rank(beta)  = 3

====================
CASE 6
phi_rank(alpha) = 3
phi_rank(beta)  = 3

====================
CASE 7
phi_rank(alpha) = 3
phi_rank(beta)  = 3

====================
CASE 8
phi_rank(alpha) = 3
phi_rank(beta)  = 3

====================
CASE 9
phi_rank(alpha) = 3
phi_rank(beta)  = 3

====================
CASE 10
phi_rank(alpha) = 3
phi_rank(beta)  = 3

====================
CASE 11
phi_rank(alpha) = 3
phi_rank(beta)  = 3

====================
CASE 12
phi_rank(alpha) = 3
phi_rank(beta)  = 3
```

```
====================
CASE 13
phi_rank(alpha) = 3
phi_rank(beta)  = 3

====================
CASE 14
phi_rank(alpha) = 3
phi_rank(beta)  = 3

CASE 15: skipped

====================
CASE 16
phi_rank(alpha) = 3
phi_rank(beta)  = 3

====================
CASE 17
phi_rank(alpha) = 3
phi_rank(beta)  = 3

====================
CASE 18
phi_rank(alpha) = 3
phi_rank(beta)  = 3

====================
CASE 19
phi_rank(alpha) = 3
phi_rank(beta)  = 5

CASE 20: skipped
```

```
def hitchin_star_alpha_transpose(alpha):

    K = hitchin_operator(alpha)

    Lambda3K = exterior_power_3(K.transpose())

    v = alpha_to_vector(alpha)

    image = Lambda3K * v

    return vector_to_alpha(image)

lam = hitchin_lambda(omega)
mu3 = lam^(3/2)

Kstar = hitchin_star_alpha_transpose(omega)

alpha2 = vector_to_alpha(
    (mu3*alpha_to_vector(omega)
     + alpha_to_vector(Kstar))/(2*mu3)
)

beta2 = vector_to_alpha(
```

```
    (mu3*alpha_to_vector(omega)
     - alpha_to_vector(Kstar))/(2*mu3)
)
```

```
beta = {
    (0,1,3):-1/3,
    (0,3,5):1.0,
    (1,3,4):1.0,
    (3,4,5):3.0
}

print(is_decomposable(beta)) # Actually if it is 0.333333333 in place of -1/3 i
get non decomposable
```

Out[37]: True

```
def hitchin_decomposition_negative(omega):

    lam = hitchin_lambda(omega)

    if lam >= -EPS:
        raise ValueError("Need lambda < 0")

    mu = sqrt(-lam)
    mu3 = mu^3

    KstarOmega = hitchin_star_alpha(omega)

    vOmega = alpha_to_vector(omega)
    vK     = alpha_to_vector(KstarOmega)

    alpha_complex = (
        vOmega + (I/mu3)*vK
    ) / 2

    alpha_bar = (
        vOmega - (I/mu3)*vK
    ) / 2

    return alpha_complex, alpha_bar
```

```
test_cases_negative = [
    # --- Category A: Standard Normal Forms & Sign Flips ---
    # Case 1 (Standard Negative Normal Form)
    {(1, 2, 3): 1.0, (1, 4, 5): -1.0, (2, 4, 6): -1.0, (3, 5, 6): -1.0},

    # Case 2 (Global Sign Flip of Standard Form)
    {(1, 2, 3): -1.0, (1, 4, 5): 1.0, (2, 4, 6): 1.0, (3, 5, 6): 1.0},

    # Case 3 (Alternative Real Splitting Orbit Variant)
    {(1, 2, 3): 1.0, (4, 5, 6): -1.0},

    # Case 4 (Permuted Split Form)
```

```
{(1, 2, 4): 1.0, (3, 5, 6): -1.0},

# Case 5 (Permuted Split Form 2)
{(1, 3, 5): 1.0, (2, 4, 6): -1.0},

# Case 6 (Permuted Standard Negative Form)
{(1, 2, 4): 1.0, (1, 3, 5): -1.0, (2, 3, 6): -1.0, (4, 5, 6): -1.0},

# Case 7 (Interleaved Permutation)
{(1, 3, 6): 1.0, (2, 4, 5): -1.0},

# --- Category B: Scaled & Weighted Intersections ---
# Case 8
{(1, 2, 3): 2.0, (4, 5, 6): -3.0},

# Case 9
{(1, 2, 4): 5.0, (3, 5, 6): -2.0},

# Case 10 (Scaled Standard Negative Form)
{(1, 2, 3): 2.0, (1, 4, 5): -2.0, (2, 4, 6): -2.0, (3, 5, 6): -2.0},

# Case 11 (Asymmetrically Scaled Standard Negative Form)
{(1, 2, 3): 3.0, (1, 4, 5): -1.5, (2, 4, 6): -1.5, (3, 5, 6): -3.0},

# Case 12
{(1, 4, 5): 0.5, (2, 3, 6): -2.0},

# --- Category C: Linear Transformations & Mixed Terms ---
# Case 13
{(1, 2, 3): 1.0, (1, 2, 4): 1.0, (4, 5, 6): -1.0},

# Case 14
{(1, 2, 3): 1.0, (1, 5, 6): -1.0, (4, 5, 6): -1.0},

# Case 15
{(1, 2, 5): 1.0, (1, 2, 6): 1.0, (3, 4, 5): -1.0, (3, 4, 6): -1.0},

# Case 16
{(1, 2, 3): 2.0, (1, 4, 5): 1.0, (4, 5, 6): -2.0},

# Case 17
{(1, 2, 3): 1.0, (1, 2, 4): -1.0, (3, 4, 5): 1.0, (4, 5, 6): -1.0},

# --- Category D: Highly Dense / Random Setup ---
# Case 18
{(1, 2, 3): 1.0, (1, 2, 4): 1.0, (1, 3, 5): -1.0, (4, 5, 6): -1.0},

# Case 19
{(1, 2, 3): 3.0, (1, 4, 6): 1.0, (2, 4, 5): -1.0, (4, 5, 6): -3.0},

# Case 20 (Highly Dense Negative Stable Form)
{
    (1, 2, 3): 1.0,
    (1, 2, 4): 1.0,
    (1, 4, 5): -1.0,
    (1, 5, 6): -1.0,
    (2, 3, 4): -1.0,
```

```
        (4, 5, 6): -1.0
    }
]

for i, case in enumerate(test_cases_negative, start=1):

    omega = convert_form(case)

    lam = hitchin_lambda(omega)

    if lam >= 0:
        continue

    alpha, alpha_bar = hitchin_decomposition_negative(omega)

    alpha_form = vector_to_alpha_complex(alpha)
    alpha_bar_form = vector_to_alpha_complex(alpha_bar)

    print("\nCASE", i)
    print("lambda =", lam)

    print(
        "alpha decomposable =",
        is_decomp(alpha_form)
    )

    print(
        "alpha_bar decomposable =",
        is_decomposable(alpha_bar_form)
    )

    print("-"*60)
```

Out[39]:

```
============================================================
CASE 1
lambda = -4.00000000000000

alpha
(0.500000000000000, 0.500000000000000*I, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.500000000000000*I, 0.000000000000000, -0.500000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.500000000000000*I, 0.000000000000000, -0.500000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000, -0.500000000000000, -0.500000000000000*I)

alpha_bar
(0.500000000000000, -0.500000000000000*I, 0.000000000000000, 0.000000000000000,
0.000000000000000, -0.500000000000000*I, 0.000000000000000, -0.500000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
-0.500000000000000*I, 0.000000000000000, -0.500000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000, -0.500000000000000, 0.500000000000000*I)

reconstruction error = 0.000000000000000

============================================================
CASE 2
lambda = -4.00000000000000
```

```
alpha
(-0.500000000000000, -0.500000000000000*I, 0.000000000000000, 0.000000000000000,
0.000000000000000, -0.500000000000000*I, 0.000000000000000, 0.500000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
-0.500000000000000*I, 0.000000000000000, 0.500000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000, 0.500000000000000, 0.500000000000000*I)

alpha_bar
(-0.500000000000000, 0.500000000000000*I, 0.000000000000000, 0.000000000000000
```

```
import numpy as np
from itertools import combinations

def get_basis_mapping():
    """Generates a stable, sorted list of all 20 combinations of 3 indices
    chosen from {0, 1, 2, 3, 4, 5}."""
    return list(combinations(range(6), 3))

def vector_to_dict_form(v, tol=1e-15):
    """Converts a flat 20-dimensional NumPy array into a dictionary
representation
    mapping sorted index tuples to their non-zero scalar coefficients."""
    basis = get_basis_mapping()
    omega_dict = {}
    for idx, coeff in enumerate(v):
        if abs(coeff) > tol:
            omega_dict[basis[idx]] = coeff
    return omega_dict

def check_plucker_decomposability(v, tol=1e-12):
    """
    Checks Plücker relations for a 3-form using the foundational definition:
    For every set I of size 2, and J of size 4, the alternating sum must be 0.
    """
    omega = vector_to_dict_form(v, tol=1e-15)

    def get_coeff(indices):
        """Returns the coefficient of a 3-form component, tracking sign flips
        caused by sorting the indices lexicographically."""
        if len(set(indices)) < 3:
            return 0.0

        # Count inversions to find permutation parity sign
        arr = list(indices)
        inversions = 0
        for i in range(len(arr)):
            for j in range(i + 1, len(arr)):
                if arr[i] > arr[j]:
                    inversions += 1
        sign = -1 if inversions % 2 != 0 else 1

        sorted_key = tuple(sorted(arr))
        return sign * omega.get(sorted_key, 0.0)
```

```
    violations = []

    # Loop over all possible index sets I (size 2) and J (size 4) from
{0,1,2,3,4,5}
    for I_tuple in combinations(range(6), 2):
        for J_tuple in combinations(range(6), 4):
            I = list(I_tuple)
            J = list(J_tuple)

            val = 0.0
            # Continuous generalized Plücker summation:
            # sum_{lambda=0..3} (-1)^lambda * alpha_(I + J[lambda]) * alpha_(J -
J[lambda])
            for idx, j_lambda in enumerate(J):
                alpha_indices = I + [j_lambda]
                beta_indices = [x for x in J if x != j_lambda]

                term = get_coeff(alpha_indices) * get_coeff(beta_indices)

                if idx % 2 == 0:
                    val += term
                else:
                    val -= term

            if abs(val) > tol:
                violations.append((I_tuple, J_tuple, val))

    is_decomp = len(violations) == 0
    return is_decomp, violations


# ============================================================================
# Test Cases Setup
# ============================================================================
test_cases_negative = [
    # --- Category A: Standard Normal Forms & Sign Flips ---
    {(1, 2, 3): 1.0, (1, 4, 5): -1.0, (2, 4, 6): -1.0, (3, 5, 6): -1.0},
    {(1, 2, 3): -1.0, (1, 4, 5): 1.0, (2, 4, 6): 1.0, (3, 5, 6): 1.0},
    {(1, 2, 3): 1.0, (4, 5, 6): -1.0},
    {(1, 2, 4): 1.0, (3, 5, 6): -1.0},
    {(1, 3, 5): 1.0, (2, 4, 6): -1.0},
    {(1, 2, 4): 1.0, (1, 3, 5): -1.0, (2, 3, 6): -1.0, (4, 5, 6): -1.0},
    {(1, 3, 6): 1.0, (2, 4, 5): -1.0},

    # --- Category B: Scaled & Weighted Intersections ---
    {(1, 2, 3): 2.0, (4, 5, 6): -3.0},
    {(1, 2, 4): 5.0, (3, 5, 6): -2.0},
    {(1, 2, 3): 2.0, (1, 4, 5): -2.0, (2, 4, 6): -2.0, (3, 5, 6): -2.0},
    {(1, 2, 3): 3.0, (1, 4, 5): -1.5, (2, 4, 6): -1.5, (3, 5, 6): -3.0},
    {(1, 4, 5): 0.5, (2, 3, 6): -2.0},

    # --- Category C: Linear Transformations & Mixed Terms ---
    {(1, 2, 3): 1.0, (1, 2, 4): 1.0, (4, 5, 6): -1.0},
    {(1, 2, 3): 1.0, (1, 5, 6): -1.0, (4, 5, 6): -1.0},
    {(1, 2, 5): 1.0, (1, 2, 6): 1.0, (3, 4, 5): -1.0, (3, 4, 6): -1.0},
    {(1, 2, 3): 2.0, (1, 4, 5): 1.0, (4, 5, 6): -2.0},
```

```
    {(1, 2, 3): 1.0, (1, 2, 4): -1.0, (3, 4, 5): 1.0, (4, 5, 6): -1.0},

    # --- Category D: Highly Dense / Random Setup ---
    {(1, 2, 3): 1.0, (1, 2, 4): 1.0, (1, 3, 5): -1.0, (4, 5, 6): -1.0},
    {(1, 2, 3): 3.0, (1, 4, 6): 1.0, (2, 4, 5): -1.0, (4, 5, 6): -3.0},
    {
        (1, 2, 3): 1.0,
        (1, 2, 4): 1.0,
        (1, 4, 5): -1.0,
        (1, 5, 6): -1.0,
        (2, 3, 4): -1.0,
        (4, 5, 6): -1.0
    }
]

# ======================================================================
# Main Execution Evaluation Loop
# ======================================================================
for i, case in enumerate(test_cases_negative, start=1):

    omega = convert_form(case)
    lam = hitchin_lambda(omega)

    if lam >= 0:
        continue

    # Decompose the form into its components
    alpha, alpha_bar = hitchin_decomposition_negative(omega)

    # Cleanly unpack only the True/False boolean result from our checker
    alpha_is_decomposable, _ = check_plucker_decomposability(alpha, tol=1e-12)
    alpha_bar_is_decomposable, _ = check_plucker_decomposability(alpha_bar,
tol=1e-12)

    print("\nCASE", i)
    print(f"lambda = {lam:.4f}")
    print("alpha decomposable     =", alpha_is_decomposable)
    print("alpha_bar decomposable =", alpha_bar_is_decomposable)
    print("-" * 60)
```

Out[59]:

```
CASE 1
lambda = -4.0000
alpha decomposable     = True
alpha_bar decomposable = True
------------------------------------------------------------

CASE 2
lambda = -4.0000
alpha decomposable     = True
alpha_bar decomposable = True
------------------------------------------------------------

CASE 6
lambda = -4.0000
```

```
alpha decomposable     = True
alpha_bar decomposable = True
------------------------------------------------------------

CASE 10
lambda = -64.0000
alpha decomposable     = True
alpha_bar decomposable = True
------------------------------------------------------------

CASE 11
lambda = -81.0000
alpha decomposable     = True
alpha_bar decomposable = True
------------------------------------------------------------
```